Le Quai Antique

Chef Arnaud Michant — Chambéry, Savoie

Documentation technique

Thomas Bruston

Titre Professionnel DWWM — Niveau 5

Organisme de formation Studi

Session juin-juillet 2026

1. Réflexions initiales technologiques

Contexte et contraintes

Le Quai Antique est une application web fullstack développée sans framework, conformément aux exigences du titre professionnel DWWM. Le projet couvre l'ensemble du cycle de développement : conception, développement, sécurisation, déploiement et documentation.

Choix de la stack technique

PHP 8.3 orienté objet a été retenu pour le back-end. L'absence de framework impose une architecture MVC construite de zéro, avec un Router maison, des Repository basés sur PDO, des Service de logique métier et des entités PHP encapsulant les données.

MySQL 8.0 a été choisi pour la base de données relationnelle. Il stocke l'ensemble des données métier : utilisateurs, réservations, carte, menus et galerie.

MongoDB 7.0 a été intégré pour les messages de contact. Ces messages sont autonomes, sans contrainte relationnelle avec les tables MySQL, ce qui justifie pleinement l'usage d'une base document. Ils sont accessibles en lecture depuis l'espace administrateur.

Docker et Docker Compose ont été adoptés pour garantir la reproductibilité de l'environnement entre le développement et la production. Un fichier docker-compose.yml couvre le développement (avec phpMyAdmin), un fichier docker-compose.prod.yml couvre la production (sans phpMyAdmin, avec redémarrage automatique).

PHPMailer 6.x est l'unique dépendance Composer du projet. Elle gère l'envoi optionnel d'emails transactionnels.

2. Configuration de l'environnement de travail

Outils utilisés

- **Visual Studio Code** — éditeur principal (extensions : PHP Intelephense, Docker,
- **Docker Desktop** — gestion des conteneurs en local
- **phpMyAdmin** — administration de la base MySQL en développement
- **MongoDB Compass** — consultation des messages de contact en développement
- **Git / GitHub** — versionnage avec Git Flow
- **Trello** — suivi des tâches sous forme de tableau Kanban
- **Figma** — conception des wireframes et maquettes

Prérequis techniques

- Docker Desktop (version 4.x ou supérieure)
- Git
- Un navigateur moderne (Chrome, Firefox, Edge)

Lancer l'environnement de développement

1. Cloner le projet
git clone https://github.com/thomas-bruston/quai-antique.git
cd quai-antique

2. Créer le fichier d'environnement
cp .env.example .env

3. Lancer les conteneurs
docker compose up -d

4. Installer les dépendances
docker compose exec app composer install

5. Initialiser la base de données
docker compose exec -T mysql mysql -u root -prootpassword < sql/create.sql
docker compose exec -T mysql mysql -u root -prootpassword \
--default-character-set=utf8mb4 quai_antique < sql/seed.sql

6. Accéder à l'application

http://localhost:8080

3. Modèle Conceptuel de Données (MCD)

(disponible en annexe)

Le MCD comporte **8 tables** organisées autour des entités métier principales.

Règles d'intégrité : toutes les clés étrangères sont déclarées avec ON UPDATE. La colonne email de la table user est déclarée UNIQUE. Les prix sont stockés en INT UNSIGNED
(centimes écartés : prix entiers en euros).
Les mots de passe sont hashés avec password_hash() (algorithme bcrypt).

4. Diagramme de classes

(disponible en annexe)

Les entités PHP reflètent directement la structure de la base de données. Chaque entité encapsule ses propriétés en private et n'expose ses données qu'à travers des getters et setters. Aucune logique métier n'est placée dans les entités, celle-ci est déléguée aux classes Service.
Les Repository étendent tous AbstractRepository qui injecte la connexion PDO. Les Controller étendent Controller qui fournit les méthodes de rendu et de redirection. Le ReservationService est indépendant de tout Repository.

5. Diagramme d'utilisation

(disponible en annexe)

L'application expose trois acteurs aux droits cumulatifs : **Visiteur**, **Client** et **Administrateur**.
Le **visiteur** accède aux fonctionnalités publiques : page d'accueil, carte, galerie, formulaire de contact, connexion et inscription. Il peut consulter le formulaire de réservation mais ne peut pas le valider.
Le **client** hérite du visiteur et accède en plus à : validation d'une réservation, consultation et gestion de ses réservations, modification de son profil, suppression de son compte.
L'**administrateur** dispose d'un espace dédié avec accès aux paramètres du restaurant, à toutes les réservations (filtre par jour), au CRUD galerie, au CRUD carte (catégories, plats, menus) et aux messages de contact (MongoDB).

6. Diagramme de séquences

(disponible en annexe)

Le diagramme illustre le parcours de réservation d'une table, de la sélection du service jusqu'à la confirmation en base de données.
1. Le client sélectionne une date et un service (midi ou soir).
2. Une requête AJAX (fetch()) est envoyée vers /reservation/check-availability.
3. Le **ReservationService** vérifie que la date est entre mardi et dimanche, génère les créneaux de 15 minutes et calcule le total de convives existants pour chaque créneau avec le **ReservationRepository**.
4. Les créneaux disponibles sont renvoyés en JSON et affichés sans rechargement de page.
5. Le client soumet le formulaire. Le serveur re-valide la disponibilité, vérifie le token CSRF et insère la réservation en base.
6. L'utilisateur est redirigé vers son tableau de bord avec un message de confirmation.

7. Annexes

MCD

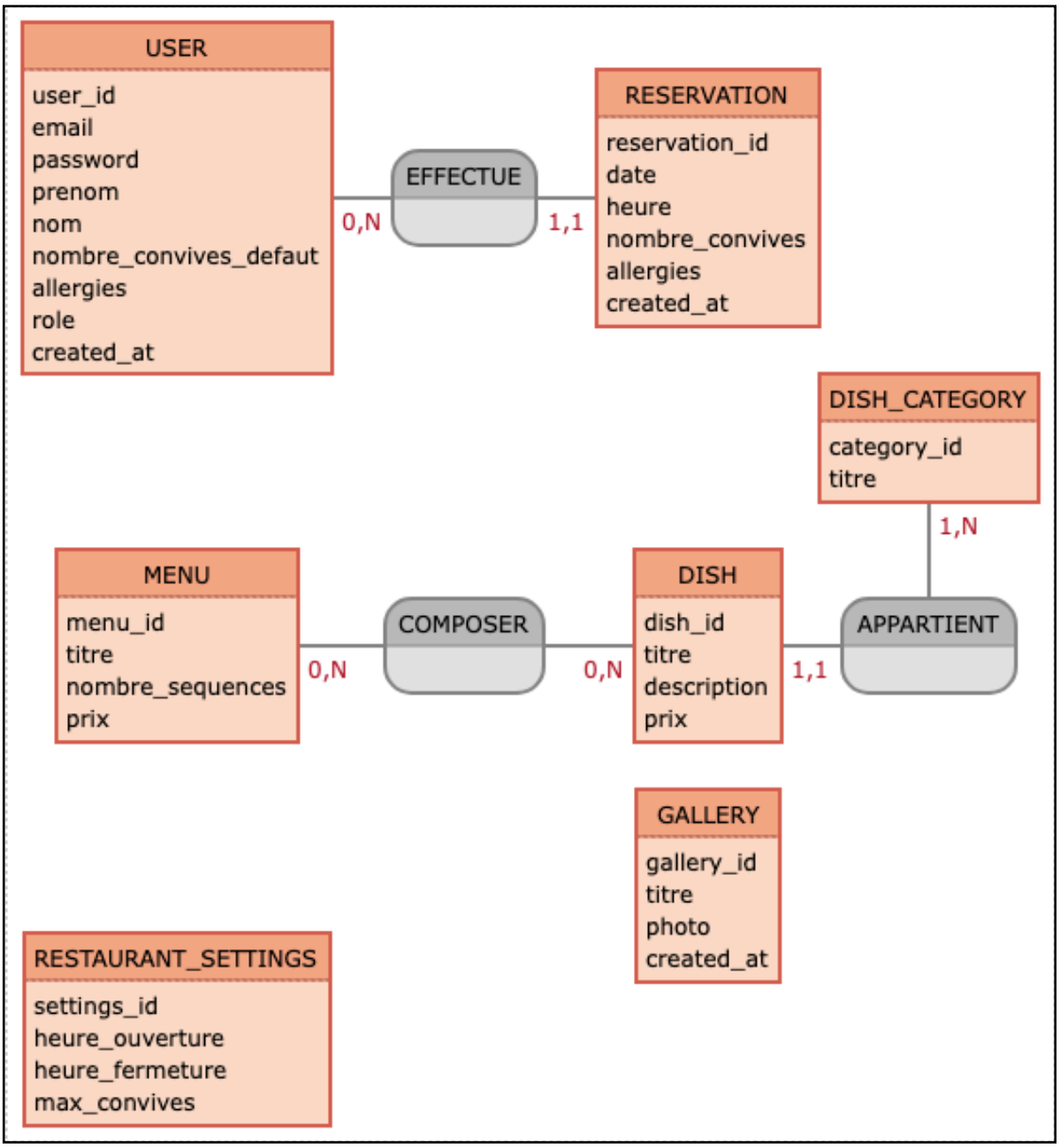

Diagramme de classe

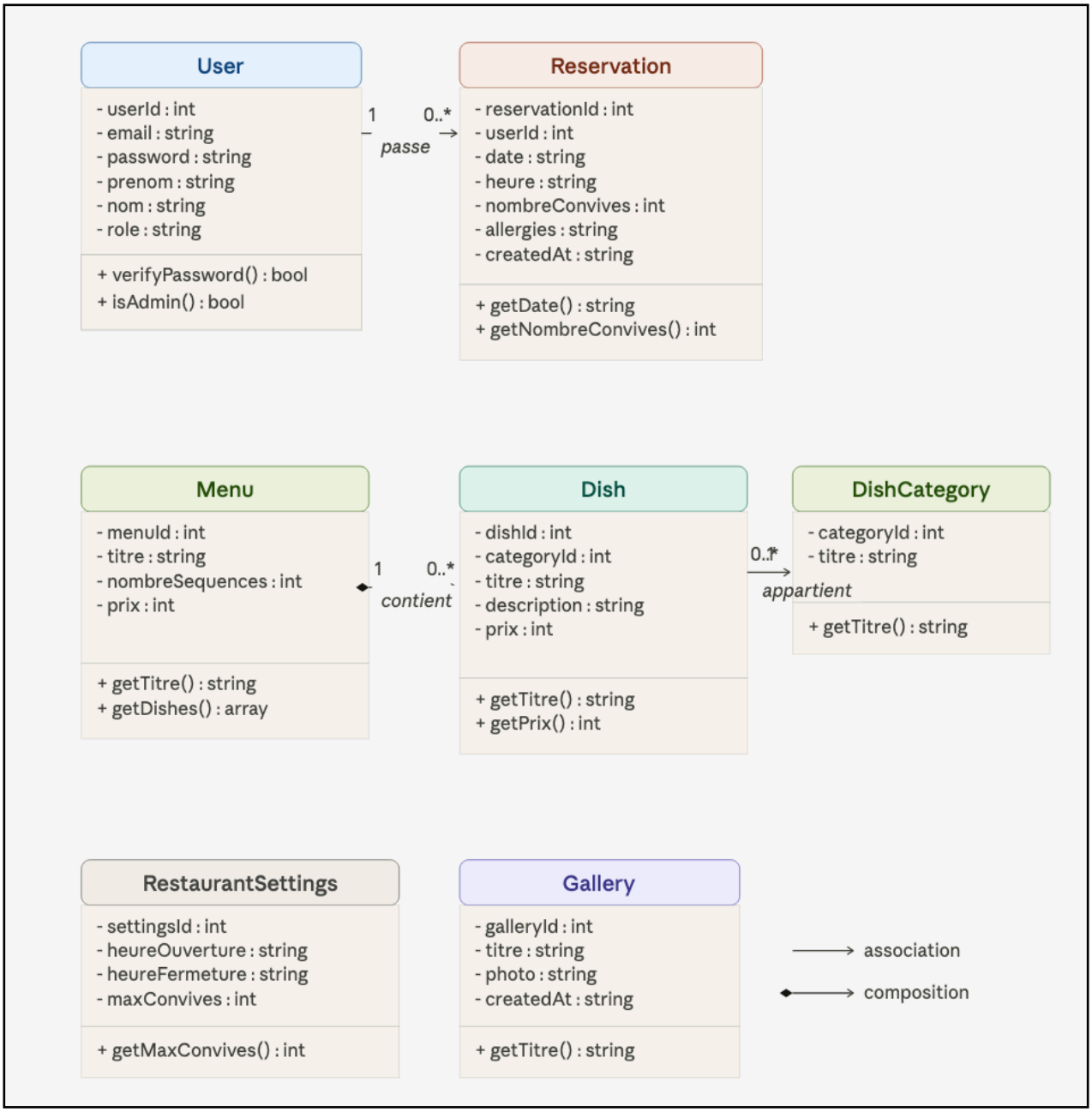

Diagramme d'utilisation

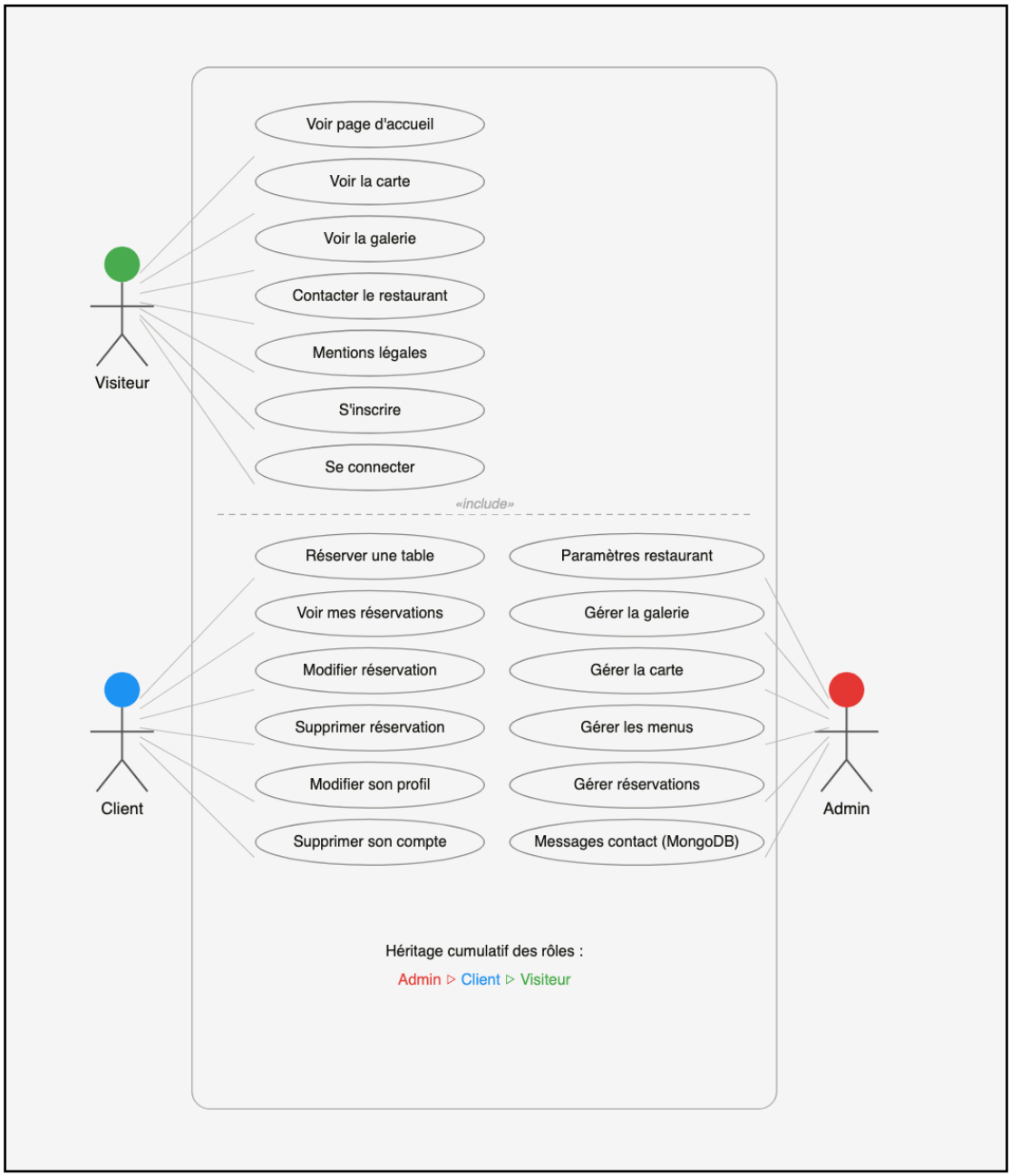
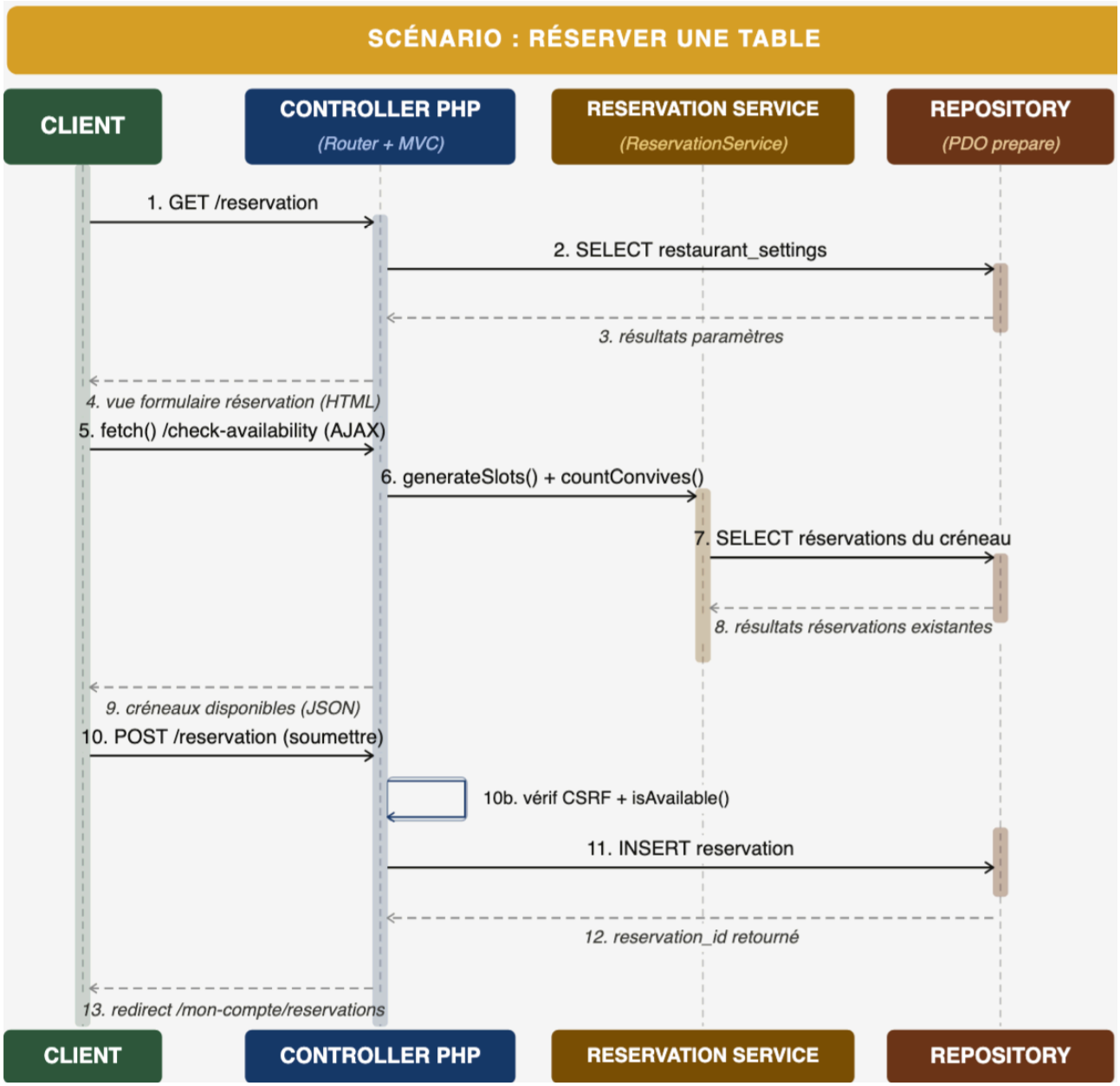
Diagramme de séquences